Relicarium II

Actividad Integradora 3

Consumo de una API REST pública

Archivo del museo

Jose Andres Mendoza Castro

Universidad ECOTEC

Desarrollo de Aplicaciones Móviles

Docente Gregorio Manuel Cruz Gonzabay

8 de noviembre de 2026

El proyecto amplía el catálogo de las actividades anteriores con una pantalla que consulta la colección del Art Institute of Chicago.

Sección I
Selección y descripción de la API pública

El Archivo del museo consulta la API del Art Institute of Chicago. Las armaduras históricas encajan con el catálogo de reliquias y permiten presentar piezas reales con título, fecha y lugar de origen. La consulta GET busca *armor*, filtra obras de dominio público y solicita hasta 12 resultados por petición.

Endpoint GET

https://api.artic.edu/api/v1/artworks/search

El enlace incluye los parámetros de la consulta usada en la app. El objeto JSON contiene la lista `data` y puede incluir `config.iiif_url`, la base para construir las direcciones de las imágenes. La siguiente muestra esquemática ilustra los campos que se leen por pieza.

```
{
  "data": [{
    "id": 123, "title": "Título de la pieza",
    "date_display": "c. 1520", "place_of_origin": "Nuremberg",
    "image_id": null
  }],
  "config": { "iiif_url": "https://www.artic.edu/iiif/2" }
}
```

Documentación de la API del Art Institute of Chicago

Sección II
Arquitectura y manejo de estados

Al abrir la pantalla, **MuseumScreen** crea un **MuseumProvider** que invoca **MuseumApiService**. El servicio se encarga de HTTP y del mapeo a **MuseumArtwork**. El Provider publica *loading*, *success* o *error* mediante `notifyListeners()`. La interfaz observa el estado con `Consumer` y muestra el indicador, las tarjetas o el botón de reintento.

Deslizar hacia abajo y pulsar Reintentar invocan el mismo método `load()`. El Provider evita consultas simultáneas y deja de notificar al cerrar la pantalla.

Sección III
Extractos de código clave

Petición HTTP y validación de la respuesta

```
final response = await http
    .get(endpoint, headers: {'Accept': 'application/json'})
    .timeout(const Duration(seconds: 10));

if (response.statusCode != 200) {
  throw http.ClientException(
    'El museo respondió con HTTP ${response.statusCode}.',
    endpoint,
  );
}
final decoded = jsonDecode(utf8.decode(response.bodyBytes));
```

El servicio espera la respuesta de forma asíncrona y limita la espera a diez segundos. La decodificación UTF-8 conserva los caracteres de los metadatos antes de interpretar el JSON.

Mapeo de los elementos de `data`

```
return data
    .map(
      (item) => MuseumArtwork.fromJson(
        item as Map<String, dynamic>,
        iiifBaseUrl: iiifBaseUrl,
      ),
    )
    .toList(growable: false);
```

`fromJson` convierte cada objeto en una pieza de Dart. La fecha, el origen y el identificador de imagen admiten valores nulos. `toJson` devuelve esos datos con las claves originales del museo.

Estados en la interfaz

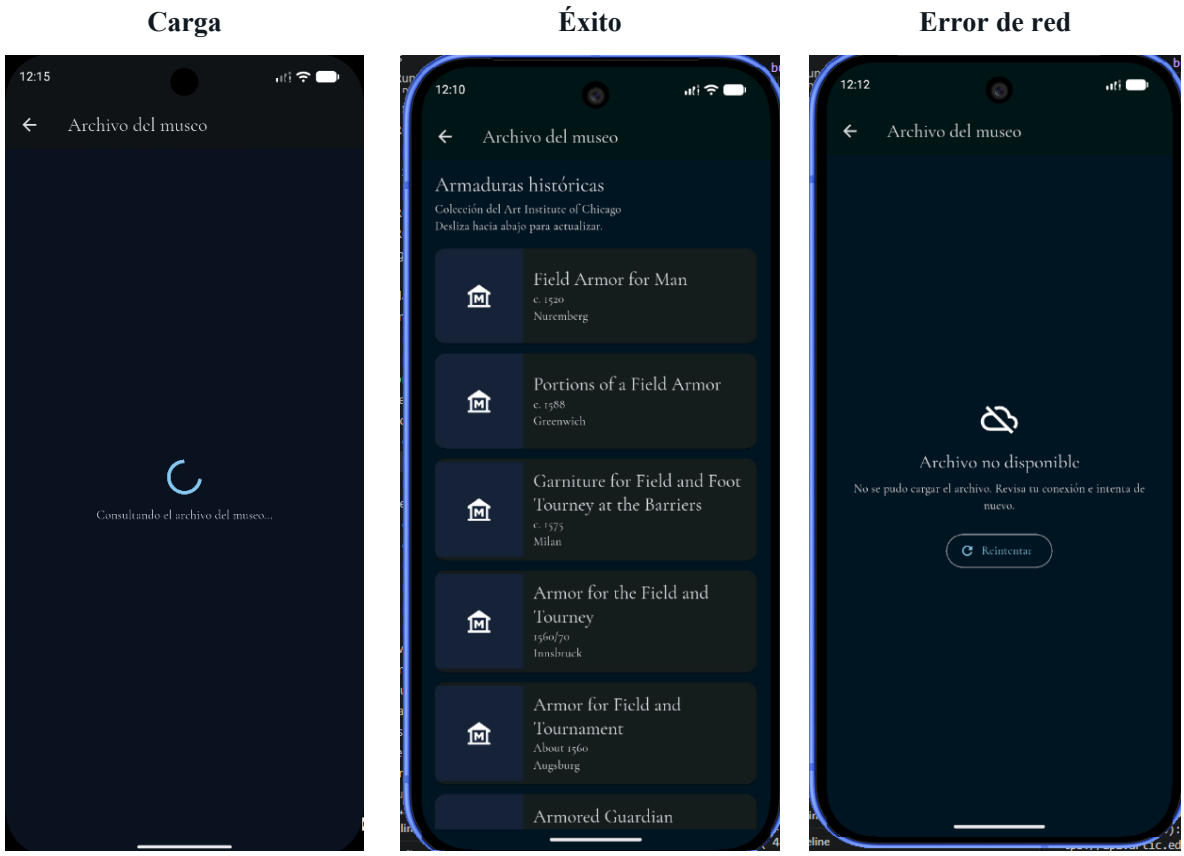
```
switch (museum.status) {
  case MuseumStatus.loading:
    return const Center(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          CircularProgressIndicator(),
          SizedBox(height: 16),
          Text('Consultando el archivo del museo…'),
        ],
      ),
    );
  // Las demás ramas muestran el error o las tarjetas.
}
```

En éxito, `RefreshIndicator` llama a `museum.load` al deslizar hacia abajo. En error, el botón Reintentar llama al mismo método.

Sección IV
Evidencias de pruebas y enlace al repositorio

Repositorio público
https://github.com/joseandresmendoza64/relicarium-flutter
Código de la Actividad 3 en feature/api-integration

Figura 1
Estados del Archivo del museo



Nota. Capturas del emulador Android Pixel 10. En la prueba de éxito, las tarjetas mostraron los metadatos del museo con el icono de reserva para las imágenes. El análisis estático con `flutter analyze` finalizó sin incidencias.

Sección V
Conclusiones

Una petición asíncrona permite que la interfaz siga respondiendo mientras espera datos. Los estados explícitos hacen visible el progreso y ofrecen una forma de recuperación ante un fallo. Separar la consulta del diseño permite ajustar el manejo de red sin modificar las tarjetas.

Se utilizó Codex como apoyo para implementar, revisar y documentar la integración. Las evidencias corresponden a la ejecución realizada por el estudiante.